

```
# 1. Crie uma função chamada calculadora que receba três parâmetros: dois
# números e uma operação (+, -, *, /).
# A função deve retornar o resultado da operação, mas precisa tratar as
# seguintes exceções:
# ☐ Divisão por zero (ZeroDivisionError)
# ☐ Tipo de dado inválido (ValueError)
```

```
def calculadora(num1, num2, operador):

    try:

        num1 = float(num1)
        num2 = float(num2)

        match operador:
            case '+':
                return num1 + num2
            case '-':
                return num1 - num2
            case '*':
                return num1 * num2
            case '/':
                if num2 == 0: # Checa se o divisor é zero
                    return "Erro: O número não pode ser dividido por 0."
                return num1 / num2
            case _:
                return "Operador inválido. Use apenas (+, -, *, /)."
```

```
except ValueError:
    print("Tipo de dado inválido")

numero1 = input('Insira um número: ')
numero2 = input('Insira um número: ')
operador = input('Insira um operador (+, -, *, /): ')

print(calculadora(numero1, numero2, operador))
```

```
↩ Insira um número: opa
Insira um número: 8
Insira um operador (+, -, *, /): -
Tipo de dado inválido
None
```

```
# 2. Crie uma função ler_inteiro() que solicita ao usuário um número inteiro.
# Se o usuário inserir um valor inválido (não inteiro), exiba uma mensagem
# e peça a entrada novamente até que um número válido seja fornecido.
```

```
def ler_inteiro():
    while True:
        entrada = input("Insira um número: ")

        try:
            num = int(entrada)

            if type(num) == int:
                break

        except ValueError:
            print("Tipo de dado inválido")
```

```
ler_inteiro()
```

```
↩ Insira um número: s
Tipo de dado inválido
Insira um número: dsad
Tipo de dado inválido
Insira um número: dwa
Tipo de dado inválido
Insira um número: 8
```

```
# 3. Crie uma função calcular_media(numeros) que recebe uma lista de
# números e retorna a média.
# Se a lista estiver vazia, a função deve tratar a exceção e exibir uma
# mensagem adequada.
```

```
lista = []
nums = [10, 5, 5, 20]

def calcular_media(numeros):
    try:
```

```
print(sum(numeros) / len(numeros))
except ZeroDivisionError:
    print('A lista esta vazia')
```

```
calcular_media(lista)
calcular_media(nums)
```

```
↵ A lista esta vazia
10.0
```

```
# 4. Crie uma função divisao_segura(a, b) que retorne o resultado da divisão
# a / b.
# Se b for zero, a função deve retornar "Erro: Divisão por zero não
# permitida!".
```

```
def divisao_segura(a, b):

    try:
        print(a / b)
    except ZeroDivisionError:
        print('Erro: Divisão por zero não permitida')
```

```
divisao_segura(2,0)
divisao_segura(4,2)
```

```
↵ Erro: Divisão por zero não permitida
2.0
```

```
# 5. Crie uma função soma_lista(numeros) que recebe uma lista de números
# e retorna a soma.
# Se a lista contiver valores inválidos, capture a exceção e exiba uma
# mensagem de erro.
```

```
lista = [10, 5, 5, 20]
lista1 = [10, 5, 5, 20, 'ola']
```

```
def soma_lista(numeros):

    try:
        print(sum(numeros))
    except TypeError:
        print('Valor invalido')
```

```
soma_lista(lista)
soma_lista(lista1)
```

```
↵ 40
Valor invalido
```

```
# 6. Crie uma função multiplicar(a, b) que retorna o produto de a e b.
# Se os valores não forem números, capture a exceção e exiba uma
# mensagem de erro.
```

```
def multiplicar(a, b):

    try:
        n1 = int(a)
        n2 = int(b)

        print(n1 * n2)

    except ValueError:
        print('Valor invalido')
```

```
multiplicar('www', 'opa')
multiplicar(2, 'opa')
multiplicar(2, 5)
```

```
↵ Valor invalido
Valor invalido
10
```

```
# 7. Crie uma função pegar_elemento(lista, indice) que retorna o elemento
# de uma lista na posição indice.
# Se o índice não existir, trate o erro.
lista1 = [10, 5, 5, 20, 'ola']
```

```
def pegar_elemento(lista, indice):  
    try:  
        print(lista[indice])  
    except IndexError:  
        print('O index é invalido')
```

```
pegar_elemento(lista1, 4)  
pegar_elemento(lista1, 7)
```

```
ola  
O index é invalido
```

```
# 8. Crie uma função contar_caracteres(texto, caractere) que conta quantas  
# vezes um caractere aparece em um texto.  
# Se texto não for uma string, exiba um erro.
```

```
def contar_caracteres(texto, caractere):  
    try:  
  
        if type(texto) != str or type(caractere) != str:  
            print('ERRO: O texto não uma string')  
            return  
  
        cont = 0  
  
        for i in texto:  
            if i == caractere:  
                cont += 1  
  
        print(cont)  
  
    except ValueError:  
        print("Valor invalido")
```

```
contar_caracteres('carro', 'r')  
contar_caracteres(2, 'r')  
contar_caracteres('carro', 2)  
contar_caracteres(5, 5)
```

```
2  
ERRO: O texto não uma string  
ERRO: O texto não uma string  
ERRO: O texto não uma string
```